Operating-System Operations and System Protection

Lecture 2 — Dual-Mode Operation, Privileged Instructions, and the Timer

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Content Not Click

Welcome: The Program That Never Yields

The big question

A student runs `while(1);` on a shared lab machine. The program executes no I/O, makes no system call, and never exits. Why does the machine not freeze — and why can the instructor still kill the process?

- The CPU has one core. Two programs cannot both be "running" at the same instant.
- If user code can execute *every* instruction, nothing stops it from disabling interrupts, reprogramming the timer, or overwriting the kernel.

Running thread for today

We follow one runaway process from its first instruction to the timer interrupt that takes the CPU away from it.

Roadmap: Four Questions About Control

1. **How does the CPU know who is running?** — the mode bit.
2. **What may user code not do?** — privileged instructions.
3. **How does the OS get the CPU back?** — the timer and preemption.
4. **How does a program ask for help?** — the system-call gate.

Every answer is a hardware mechanism, not a software convention: protection only works because the hardware enforces it.

Comments That Click

The operating system is not a remote controller. It is a program that must run on the same CPU as the code it supervises.

Concepts Click

The OS Is a Program Too

- The kernel uses the same CPU, the same registers, and the same physical memory as every user program.
- It cannot run "in the background" while user code runs; the CPU executes one instruction stream at a time.
- The kernel therefore runs only when an **event** transfers control to it: a system call, an interrupt, or an exception.
- The hardware must be able to tell, at every instant, whether the current instruction stream is *trusted* kernel code or *untrusted* user code.

The design problem

Build a machine that can run untrusted code at full speed, yet guarantee that the trusted kernel can always regain control.

What Goes Wrong Without a Boundary

- A buggy pointer writes over another process's memory — or over the kernel itself.
- A malicious program disables interrupts, and the timer can no longer fire.
- A program reprograms the timer to never expire, then loops forever.
- A program issues raw disk commands, bypassing the file system and every access-control check.

The lesson

No boundary means no isolation, no multiplexing, and no trust. The rest of this lecture is the mechanism that creates the boundary.

A single hardware bit separates the program that can do anything from the programs that must ask permission.

One Bit, Two Worlds

Dual-mode operation

A hardware **mode bit** records the current privilege level. The kernel runs in **kernel mode** (mode bit $= 0$); user programs run in **user mode** (mode bit $= 1$). Some instructions are legal only when the bit is 0.

- **Kernel mode:** full instruction set, access to all memory and all devices, can change the mode bit.
- **User mode:** restricted instruction set, access only to its own address space, cannot change the mode bit directly.
- The bit is *hardware state*, not an ordinary memory variable.

Silberschatz et al., Ch. 1 §1.5, pp. 20–23.

Who May Execute What?

Privileged (kernel only)

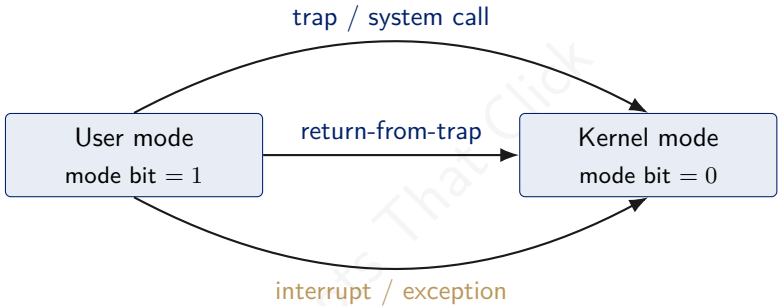
- set or clear the mode bit
- disable or enable interrupts
- load the timer
- issue raw I/O instructions
- change the page-table base
- halt the machine

Unprivileged (user or kernel)

- arithmetic and logic
- loads and stores to the process's own memory
- branches and function calls
- read the clock (not set it)
- execute a trap instruction

A privileged instruction executed in user mode does not silently fail: it traps to the kernel, which normally terminates the process.

The Mode Transition Graph



- Entry into the kernel is always a **controlled** hardware event.
- The return path restores the saved user context and the user mode bit.
- The kernel does not “call” user code; it returns to it.

The Mode Bit Is Hardware, Not a Variable

- If the mode bit were stored in ordinary memory, a user program could simply write `0` into it and become the kernel.
- The instruction that changes the bit is itself privileged, so user code cannot execute it.
- The hardware defines exactly which events may switch modes: interrupts, exceptions, and traps.
- This small, hardware-enforced boundary is the foundation of the **trusted computing base**.

Consequence

Protection is not a policy the OS can choose to enforce; it is a property the hardware guarantees. The OS decides *what* to protect, but the mode bit decides *whether* it can.

Worked Trace: One `write` Crosses the Boundary

1. User code calls the C library `write(1, buf, n)`.
2. The library places the system-call number and arguments in registers.
3. It executes a **trap** instruction (`syscall` on x86-64).
4. Hardware switches to kernel mode, saves the user PC and flags, and jumps to the trap handler.
5. The kernel validates the arguments, performs the I/O, and stores a return value.
6. A **return-from-trap** restores the user context and the user mode bit.
7. The library returns the result; on failure it sets `errno`.

We will open this path up in Lecture 3. For now, notice that the mode switch is a hardware event, not a function call.

Content Thief Sick

Socratic Check 1

Question

Why can't the mode bit live in ordinary memory? Describe the attack that becomes possible if it does.

- A user program could store `0` into the mode variable.
- It could then disable interrupts, reprogram the timer, or overwrite the page tables.
- The hardware would no longer have a trusted state to return to.

The bit must be outside the address space that user code can write.

Dual-mode protects memory and devices. It does not protect time: a user program can loop forever without executing a single privileged instruction.

The Loop That Never Leaves User Mode

- `while(1);` executes only unprivileged instructions.
- It never traps, never calls the OS, and never touches a device.
- If nothing external interrupts it, the kernel never runs again.
- The machine is not corrupted — it is simply monopolized.

The missing mechanism

The OS needs an event that fires *without* the user program's cooperation. That event is the **timer interrupt**.

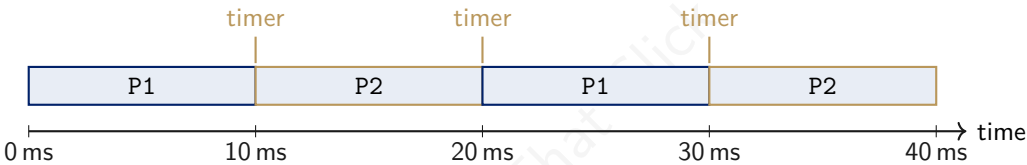
The Timer Is the External Event

- A hardware timer counts down independently of the instruction stream.
- When it reaches zero, it raises a **timer interrupt**.
- The interrupt switches the CPU to kernel mode and jumps to the kernel's timer handler.
- The handler saves the running process's context and calls the scheduler.
- Loading the timer is a **privileged** instruction: user code cannot set it.

The timer is a device, not a software alarm. If user code could reload it, preemption would be optional — and a malicious program would opt out.

Concepts That Click

A Quantum on the Timeline



- Each process receives one 10 ms **quantum**.
- The timer fires at the end of every quantum, even if the process is still runnable.
- The scheduler decides who runs next; the timer only guarantees that the decision happens.

Concept That Click

Timer + Scheduler = Preemption

Preemption

The kernel forcibly takes the CPU away from a running process at a timer interrupt, saves its context, and may schedule a different process.

- **Cooperative:** a process yields voluntarily — one infinite loop stops the system.
- **Preemptive:** the timer makes yielding unnecessary — the kernel keeps control of the CPU.

Trade-off: a large quantum wastes response time; a tiny quantum wastes CPU time on context switches.

Worked Example: What Does a Quantum Cost?

Setup

A context switch costs $12\ \mu$s. The quantum is Q. What fraction of CPU time is spent switching?

Quantum Q	**Switches per second**	**Overhead**
$100\ \mu$s	$10{,}000$	$120{,}000\ \mu\text{s} = 12\%$
1 ms	$1{,}000$	$12{,}000\ \mu\text{s} = 1.2\%$
10 ms	100	$1{,}200\ \mu\text{s} = 0.12\%$

Concepts Not

Overhead $= \dfrac{12\ \mu\text{s}}{Q}$. The same formula explains why the quantum cannot simply be made as small as possible.

Socratic Check 2

Question

If a small quantum improves response time, why not set the quantum to 1 μs?

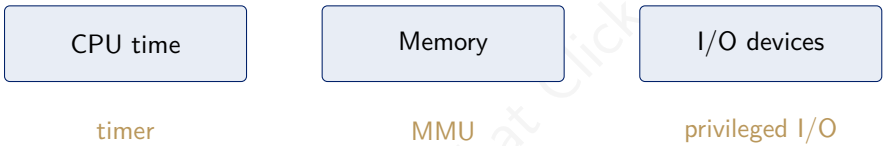
- At 1 μs, the switch overhead alone is 1200% — the CPU spends more time switching than executing.
- Cache and TLB state are destroyed on every switch.
- The scheduler itself consumes CPU time.
- Real systems choose a quantum that balances responsiveness against overhead (often 1–100 ms).

Concepts That

The mode bit and the timer protect the CPU. A complete boundary also protects memory and the devices.

Concepts Clinic

Three Things the Kernel Protects



- **CPU time:** the timer forces the kernel to run.
- **Memory:** the MMU checks every user access against the process's legal address range.
- **I/O:** devices are reachable only through privileged instructions and kernel drivers.

Concept That Click

Memory Protection in One Slide

- **Contiguous allocation:** base and limit registers bound the process; the hardware checks every address.
- **Paging:** the page table maps only the process's legal pages; an unmapped page traps to the kernel.
- **Segmentation:** base and limit registers per segment; the hardware checks the offset.

The common pattern

User code never sees a physical address. Every access is translated and checked by hardware; a violation becomes a **trap**, not a silent write.

Details in Unit III (Lectures 20–25).

I/O Protection

- All raw I/O instructions are privileged: a user program cannot talk to a disk, network card, or timer directly.
- The user-level `read`/`write` calls are system calls; the kernel validates the buffer, the device, and the count.
- The kernel then invokes the device driver, which programs the controller.
- `cat` never touches the disk controller — it asks the kernel, and the kernel decides whether the request is legal.

Why this matters

Direct device access would let a process bypass file permissions, corrupt another process's data, or hang the device for everyone.

Protection vs. Security

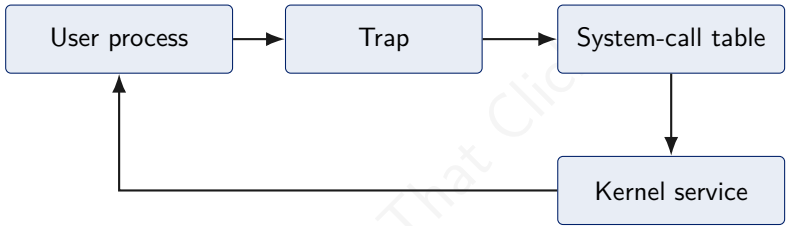
Protection	**Security**
Internal mechanisms among processes	External threats and attackers
memory isolation, privilege levels, access control	authentication, malware defense, network firewalls
Enforced by the OS and hardware	Enforced by policy, cryptography, and administrative practice

- Protection is a *mechanism*; security is a *policy* that uses the mechanism.
- A perfect mode bit does not help if the administrator grants every user root access.

Stallings, Ch. 3 §3.6, p. 143.

Concepts Tied Click

The System-Call Gate



- The trap is the **only legal door** from user mode into the kernel.
- The kernel validates every argument and every pointer before acting.
- The return path restores user mode; the kernel never returns to arbitrary user code.

Lecture 3 opens this door and walks through it.

Courses That Click

Socratic Check 3

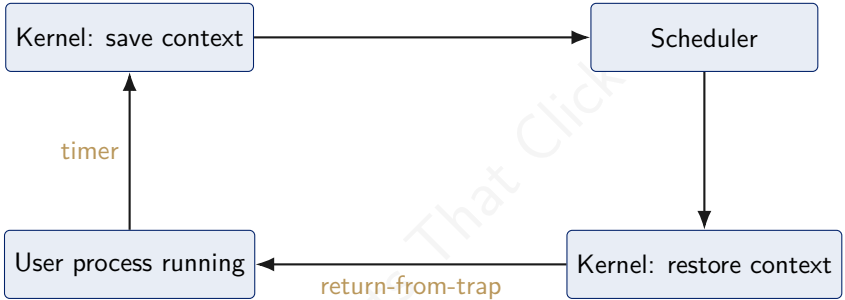
Question

If all I/O instructions are privileged, how does `cat` read a file? Trace the legal path in three steps.

1. `cat` calls the library `read`, which traps into the kernel.
2. The kernel checks the file descriptor, buffer, and count, then asks the device driver to perform the transfer.
3. The driver programs the controller; the kernel returns the data and restores user mode.

Protection is not one mechanism. It is a loop: the timer fires, the kernel runs, the scheduler chooses, and user code resumes — or traps when it asks for something it cannot do.

The Full Control Loop



- The loop runs on every timer tick and every blocking system call.
- The kernel is *event-driven*: it runs because something happened, not because it is "always on".
- The user process cannot see the loop; it only sees time passing.

Worked Trace: Stopping the Runaway Process

1. $t = 0$: `P1` enters `while(1);` in user mode.
2. $t = 10$ ms: the timer fires; hardware switches to kernel mode.
3. The kernel saves `P1`'s PC and registers in its PCB.
4. The scheduler picks `P2`; the kernel restores `P2`'s context and returns to user mode.
5. $t = 20$ ms: the timer fires again; the scheduler may pick `P1` or a third process.
6. If `P1` must be stopped, the kernel can deliver a signal or terminate it — it already has control.

No cooperation from `P1` is required at any step.

Concept That Click

Common Misconceptions

Wrong

- "Dual-mode is a software convention."
- "The timer is a software alarm."
- "User code can disable interrupts if it is fast enough."
- "The OS runs continuously in the background."

Right

- The mode bit and privileged instructions are hardware.
- The timer is a hardware device loaded by the kernel.
- Interrupt control is privileged; attempts trap.
- The kernel is event-driven, entered through traps and interrupts.

Takeaways

- **Dual-mode operation** uses a hardware mode bit to separate trusted kernel code from untrusted user code.
- **Privileged instructions** protect the mode bit, the timer, the page tables, and the devices.
- **The timer** is the external event that lets the kernel preempt a runaway process.
- **The system-call gate** is the controlled entry point; the kernel validates every request.
- Protection is a *hardware-enforced* boundary, not a promise the OS makes.

Concepts That Click

Next: System Calls

- Today: how the OS stays in control — mode bit, privileged instructions, timer, and the trap gate.
- Next lecture: what happens *inside* the gate — how a program names a service, how parameters travel, and what kinds of calls exist.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 2 §2.3–2.4 [2]; Bovet & Cesati, *Understanding the Linux Kernel*, Ch. 10 [1].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 1 §1.5 (dual-mode operation, timer, protection, pp. 20–23).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 §3.5 (execution of the OS, p. 140), §3.6 (security issues, p. 143).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 4 (interrupts and exceptions, pp. 131–188), Ch. 10 (system calls, pp. 398–419), for the concrete x86/Linux view.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Computer Hut Click

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.

Concepts That Click